

Dokumentasi Teknis: AI Landing Page Generator

Kepada: Tim Developer

Tanggal: 23 Juli 2025

Subjek: Dokumentasi Teknis & Peta Jalan Pengembangan untuk Proyek Generator Landing Page Berbasis AI

1. Ringkasan Eksekutif

Dokumen ini merinci arsitektur teknis dan alur kerja untuk sistem generator landing page. Tujuan utama proyek ini adalah menciptakan "**Pabrik Konten**" yang berjalan secara terpisah (decoupled) dari platform WordPress utama. Sistem ini akan secara otomatis menghasilkan potongan HTML modular yang diisi oleh AI (Google Gemini) dan data statis (CSV), yang kemudian dapat dipublikasikan ke WordPress.

Arsitektur ini dipilih untuk memisahkan proses pembuatan konten yang kompleks dari website live, sehingga menjaga stabilitas, keamanan, dan memungkinkan skalabilitas di masa depan.

2. Arsitektur Sistem

Sistem ini terdiri dari dua lingkungan yang terpisah:

1. Pabrik Konten (Content Factory):

- **Lingkungan:** Berjalan di lingkungan lokal atau server terpisah.
- **Tanggung Jawab:** Mengeksekusi skrip Python untuk berkomunikasi dengan AI, memproses data, merender template Jinja2, dan menghasilkan file output (.html dan .json).
- **Teknologi:** Python, Jinja2, Google Gemini API.

2. Toko (Storefront):

- **Lingkungan:** Website WordPress live.
- **Tanggung Jawab:** Menampilkan konten HTML yang sudah jadi. Mengelola aset global (CSS/JS), header, footer, dan fungsionalitas inti situs.
- **Teknologi:** WordPress, Blocksy Child Theme.

3. Tumpukan Teknologi

- **Bahasa Pemrograman:** Python 3.x
- **Templating Engine:** Jinja2
- **AI Model:** Google Gemini API (model gemini-1.5-flash-latest)
- **Frontend Framework:** Tailwind CSS (digunakan dalam template HTML sumber)

- Manajemen Data Statis: CSV
- Target CMS: WordPress

4. Alur Kerja Teknis

Tahap 1: Persiapan Aset & WordPress (Setup Satu Kali)

1. **Ekstraksi Komponen Modular:**
 - o File HTML dari template sumber (Sandbox) dipecah menjadi komponen-komponen logis (hero.jinja, services.jinja, faq.jinja, dll.).
 - o Setiap komponen disimpan di templates/components/ dan semua teks statis diganti dengan placeholder Jinja2 (contoh: {{ hero.headline }}).
2. **Optimasi CSS:**
 - o Jalankan npm run build pada direktori template sumber untuk menghasilkan file style.css yang teroptimasi dan ringan.
3. **Setup WordPress Child Theme:**
 - o Buat dan aktifkan Blocksy Child Theme.
 - o Salin seluruh folder assets (css, js, fonts, img) dan file style.css yang sudah dioptimalkan ke dalam direktori Child Theme.
 - o Daftarkan semua aset CSS dan JS melalui functions.php menggunakan wp_enqueue_style() dan wp_enqueue_script(). **Ini adalah langkah krusial** untuk memastikan path aset selalu benar dan tidak ada konflik.

Tahap 2: Proses Generasi Konten (Eksekusi Skrip Python)

Proses ini diorkestrasi oleh main.py dan mengikuti alur: **AI = (JSON + CSV) -> HTML**.

1. **Input:** Developer memberikan **topik** utama untuk halaman yang akan dibuat.
2. **Panggilan AI (ai_generator.py):**
 - o Sebuah *prompt* yang sangat terstruktur dibuat. Prompt ini secara eksplisit mendefinisikan skema JSON yang diharapkan sebagai output, termasuk nama-nama kunci dan tipe data.
 - o Permintaan dikirim ke Gemini API. AI mengembalikan konten dinamis (teks, judul) dalam format JSON.
3. **Penggabungan Data (data_manager.py & main.py):**
 - o Fungsi di data_manager.py membaca file data_aset.csv dan mengubahnya menjadi dictionary Python untuk pencarian cepat.
 - o Di main.py, skrip melakukan iterasi pada data JSON dari AI. Untuk setiap item yang memiliki id (contoh: service_id), skrip akan mencari id tersebut di data CSV dan menggabungkan data statis (link, path gambar) ke dalam objek data dari AI.
4. **Render HTML (template_renderer.py):**

- Data gabungan yang sudah lengkap dikirim ke fungsi render.
- Fungsi ini menggunakan Jinja2 untuk merender template master (landing_page_base.jinja), yang kemudian akan me-include semua komponen yang diperlukan.
- Hasilnya adalah satu file .html (potongan konten) dan satu file .json (data yang digunakan) disimpan untuk arsip.

5. Logika Inti & Pengembangan Lanjutan

Logika Pembangunan Konten & Randomisasi

Untuk membuat halaman yang bervariasi, kita akan mengimplementasikan dua jenis logika dinamis di dalam main.py:

1. Pemilihan Komponen Berdasarkan Tipe Bisnis:

- **Prompt:** Minta AI untuk menambahkan key "business_type" (misal: "law", "health") di JSON.
- **Logika Python:**

```
business_type = page_data.get('business_type', 'default').lower()
hero_template_file = f'components/hero_{business_type}.jinja'
# Tambahkan fallback ke default jika file spesifik tidak ada
if not os.path.exists(os.path.join('templates', hero_template_file)):
    hero_template_file = 'components/hero_default.jinja'
page_data['hero_template_to_use'] = hero_template_file
```
- **Template Jinja:**

```
{% include hero_template_to_use %}
```

2. Randomisasi Gaya Komponen:

- **Persiapan:** Buat beberapa varian template untuk satu komponen (misal: service_style_1.jinja, service_style_2.jinja).
- **Logika Python:**

```
import random
service_styles = ['components/service_style_1.jinja',
'components/service_style_2.jinja']
page_data['service_template_to_use'] = random.choice(service_styles)
```
- **Template Jinja:**

```
{% include service_template_to_use %}
```

Logika Enrichment (Pemanggilan AI Kedua)

Modul `aicontent_enrichment.py` dapat diimplementasikan untuk memperdalam konten.

- **Tujuan:** Setelah mendapatkan konten dasar, kita bisa meminta AI untuk mengembangkannya.
- **Alur Kerja:**
 1. Setelah `ai_generator.py` menghasilkan JSON dasar, `main.py` akan memanggil fungsi dari `aicontent_enrichment.py`.
 2. Fungsi ini akan mengambil salah satu teks dari JSON (misal, deskripsi layanan).
 3. Ia akan membuat **prompt baru** yang lebih spesifik, contoh:
"Anda adalah seorang ahli SEO. Berdasarkan deskripsi layanan singkat ini: '[deskripsi dari panggilan pertama]', kembangkan menjadi paragraf yang lebih detail yang mencakup poin-poin tentang riset keyword, on-page SEO, dan link building. Jaga agar nada tulisan tetap profesional."
 4. Hasil dari panggilan kedua ini kemudian akan menggantikan deskripsi singkat di dalam objek `page_data` sebelum dikirim ke `template_renderer.py`.

6. Struktur Kode Python Modular yang Disarankan

```
proyek_generator/  
|  
|-- main.py          # Skrip utama untuk menjalankan generator  
|-- config.py        # Menyimpan API keys, paths, dan konfigurasi  
|-- ai_generator.py   # Membuat prompt dan memanggil Gemini API  
|-- data_manager.py   # Membaca dan memproses file CSV  
|-- template_renderer.py # Merender template Jinja2 dan menyimpan file  
|  
|-- templates/  
|   |-- landing_page_base.jinja  
|   |-- components/  
|       |-- hero.jinja  
|       |-- services.jinja  
|       `-- ...  
|  
|-- data_aset.csv     # Data statis untuk aset  
|-- output/           # Folder untuk menyimpan hasil .html dan .json  
|  
`-- (Future Development)/
```

```
|-- aicontent_enrichment.py # Untuk panggilan AI kedua  
|-- aiwordpress.py         # Untuk integrasi dengan WordPress REST API
```

7. Peta Jalan Pengembangan ke Depan

1. **Prioritas Utama - Automasi WordPress (aiwordpress.py):**
 - Implementasikan modul untuk berinteraksi dengan **WordPress REST API**.
 - Gunakan library requests di Python untuk otentikasi (Application Passwords) dan mengirim permintaan POST untuk membuat halaman baru.
 - Payload permintaan akan berisi judul halaman dan konten HTML yang sudah dirender.
 - **Tujuan Akhir:** Menjalankan main.py tidak hanya menghasilkan file, tetapi juga langsung mempublikasikan halaman sebagai *draft* di WordPress.
2. **Generasi Massal:**
 - Modifikasi main.py untuk membaca daftar topik dari sebuah file CSV.
 - Implementasikan perulangan untuk menjalankan seluruh proses generasi untuk setiap baris di CSV tersebut.
3. **Integrasi Gambar Dinamis:**
 - Kembangkan modul untuk terhubung ke API Unsplash atau Pexels.
 - Berdasarkan topik, skrip bisa otomatis mencari gambar yang relevan, mengunduhnya, dan menyisipkan path-nya ke dalam data sebelum rendering.
4. **Validasi & Error Handling Lanjutan:**
 - Gunakan library Pydantic untuk mendefinisikan model data yang ketat. Ini akan secara otomatis memvalidasi struktur JSON dari AI. Jika tidak valid, skrip bisa otomatis mencoba lagi atau memberikan log error yang lebih spesifik.